COMP3011
计算机体系结构
2.2 指令级并行与动态调度

授课教师：王强
邮箱：qiang.wang@hit.edu.cn



第2.2节 指令级并行与动态调度

1、指令级并行基本概念

2、硬件方法-Scoreboard

3、硬件方法-Tomasulo

目录

➢ 指令级并行基本概念

- 流水线与相关
- 指令调度方法

➢ 硬件方法-Scoreboard

- 技术要点
- 实例分析

➢ 硬件方法-Tomasulo

- 技术要点
- 实例分析

1、指令级并行基本概念

➢ 指令级并行（Instruction-Level Parallelism，ILP）

- 指指令之间存在的一种并行性，利用它，计算机可以并行执行两条或两条以上的指令。

➢ 开发ILP的途径有两种

- 资源重复，重复设置多个处理部件，让它们同时执行相邻或相近的多条指令；
- 采用流水线技术，使指令重叠并行执行。

➢ 本节研究

- 如何利用各种技术来开发更多的指令级并行 ,以最终提高处理器的运算能力

1、指令级并行基本概念

➢ 开发ILP的方法可以分为两大类

- ❑ 主要基于硬件的动态开发方法
- ❑ 基于软件的静态开发方法

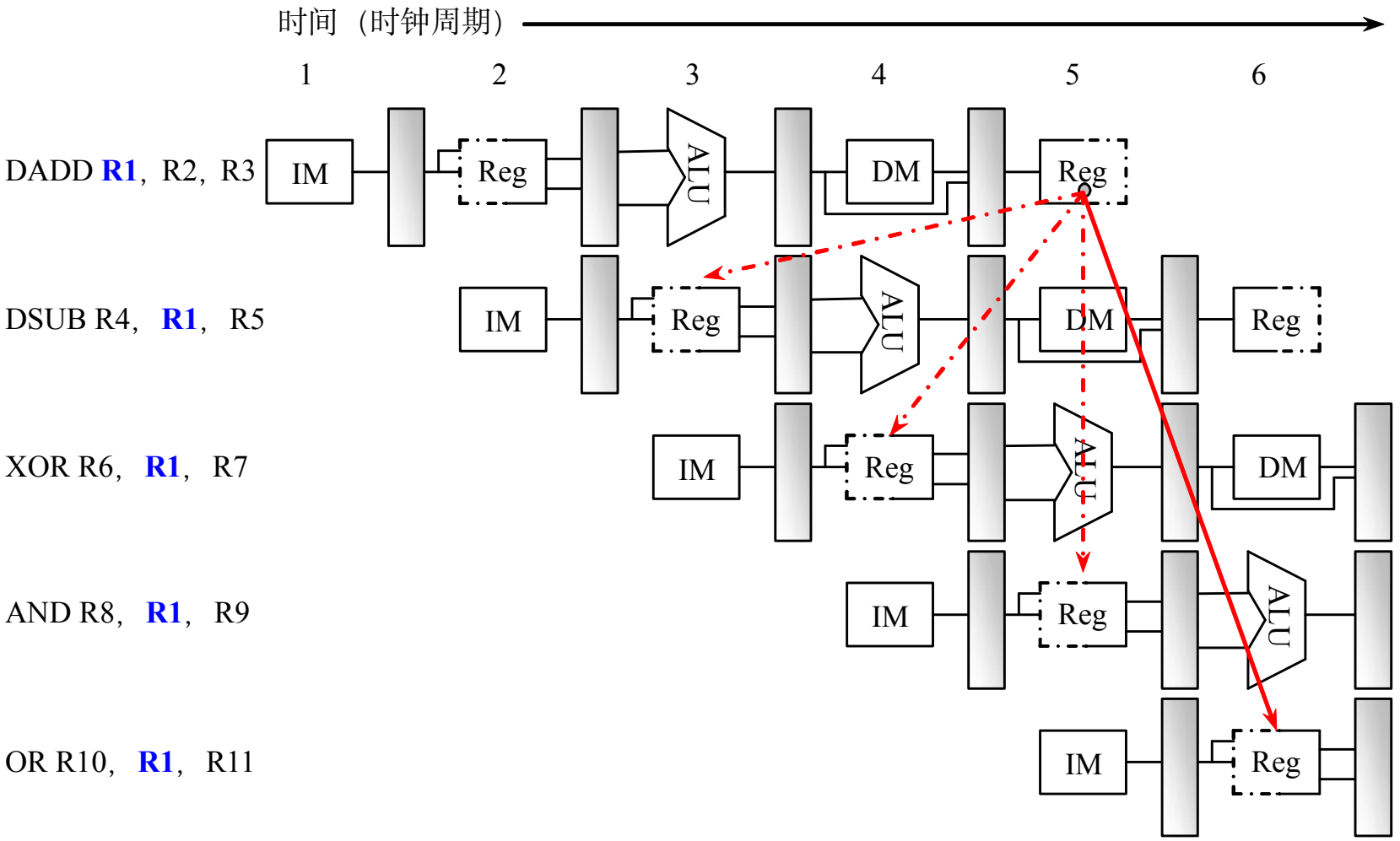
➢ 流水线处理机的实际CPI

- ❑ 理想流水线的CPI加上各类停顿的时钟周期数：

$$\text{CPI流水线} = \text{CPI理想} + \text{停顿结构冲突} + \text{停顿数据冲突} + \text{停顿控制冲突}$$

- ❑ 理想CPI是衡量流水线最高性能的一个指标。
- ❑ IPC：Instructions Per Cycle （每个时钟周期完成的指令条数）

1、指令级并行基本概念 – 流水线回顾



流水线的数据冲突举例

1、指令级并行基本概念 – 流水线示例2

- 流水线技术大大提高执行效率
  - ✓ 1913年，美国的福特公司亨利·福特开发出了世界上第一条“流水线”；
  - ✓ 每辆T型汽车的组装时间由原来的12小时28分钟缩短至90分钟，生产效率提高了8倍。



- 流水线处理机技术提高程序执行效率



@51CTO博客

1、指令级并行基本概念 - 相关

➢ 相关与流水线冲突

- ❑ 相关：两条指令之间存在某种依赖关系。
  - ○ 如果两条指令相关，则它们就有可能不能在流水线中重叠执行或者只能部分重叠执行。
- ❑ 相关有3种类型
  - ○ 数据相关（也称真数据相关）
  - ○ 名相关
  - ○ 控制相关

➢ 数据相关

- ❑ 对于两条指令*i*（在前，下同）和*j*（在后，下同），如果下述条件之一成立，则称指令*j*与指令*i*数据相关。
  - ○ **指令*j*使用指令*i*产生的结果；**
  - ○ **指令*j*与指令*k*数据相关，而指令*k*又与指令*i*数据相关。**
- ❑ 数据相关具有传递性。

数据相关反映了数据的流动关系，即如何从其产生者流动到其消费者。

例如： 下面这一段代码存在数据相关。(数组增值)

```
Loop:  L.D     F0, 0(R1)        // F0为数组元素
       ADD.D   F4, F0, F2       // 加上F2中的值
       S.D     F4, 0(R1)        // 保存结果
       DADDIU  R1, R1, #-8      // 数组指针递减8个字节
       BNE     R1, R2, Loop     // 如果R1≠R2，则分支
```

- 如果两条指令之间有数据相关，那么它们就不能同时执行或是完全重叠执行
- 同时执行这些指令会造成正在流水的处理机检测到这种冲突并插入暂停，从而减少甚至取消指令之间的重叠
- 指令序列中存在的数据相关反映出产生该指令序列的程序源代码的相关关系

相关性是程序的一个特性，
是否一个相关会导致实际的冲突，是否该冲突会造成暂停，
这是流水线结构的基本特性。

- 数据相关性会限制可以开发的指令级并行性
- 解决方法
  - **保持相关关系但避免冲突**
    - 对代码进行调度是保持相关关系并避免冲突常用的一个最基本方法
  - **通过变换代码来消除相关关系**
- 数据相关的检测
  - **当数据的流动是经过寄存器时，相关的检测比较直观和容易**
  - **当数据的流动是经过存储器时，检测比较复杂。**
    - 相同形式的地址其有效地址未必相同；
    - 形式不同的地址其有效地址却可能相同。

名相关

- 名：指令所访问的寄存器或存储器单元的名称。
- 如果两条指令使用相同的名，但是它们之间并没有数据流动，则称这两条指令存在名相关。

- 指令*j*与指令*i*之间的名相关有两种：
  - **反相关：如果指令*j*写的名与指令*i*读的名相同，则称指令*i*和*j*发生了反相关。**

指令*j*写的名 = 指令*i*读的名

必须保证指令的原来顺序，以确保*i*能读到正确的值。

  - **输出相关：如果指令*j*和指令*i*写相同的名，则称指令*i*和*j*发生了输出相关。**

指令*j*写的名 = 指令*i*写的名

操作顺序必须得到保证，以使最后写入的值得以保存

- 名相关的两条指令之间并没有数据的传送。
- 如果一条指令中的名改变了，并不影响另外一条指令的执行。
- 换名技术
  - 换名技术：通过改变指令中操作数的名来消除名相关。
  - 对于寄存器操作数进行换名称为寄存器换名。

既可以用编译器静态实现，也可以用硬件动态完成。

例如：考虑下述代码：

```
DIV.D     F2，F8，F4
ADD.D     F8，F0，F12
SUB.D     F10，F8，F14
```

DIV.D和ADD.D存在反相关。

进行寄存器换名（F8换成S）后，变成：

```
DIV.D     F2，F8，F4
ADD.D     S，F0，F12
SUB.D     F10，S，F14
```

名字相关的指令是可以同时执行或进行重新排序的

1、指令级并行基本概念 – 冲突

- 当指令之间存在相关性，而且它们的执行时间相近时，可能会在流水线中重叠操作，或者指令的重新排序改变了有相关的操作数的访问顺序，这时冲突就发生了。
- 冲突类型分为：数据冲突、结构冲突、控制冲突

程序顺序：

由源程序确定的在完全串行方式下指令的执行顺序

只有在可能会导致错误的情况下，才保持程序顺序。

数据冲突

- 根据指令在流水线中要保留的读访问和写访问的顺序，可以将数据冲突分为3种类型。

考虑两条指令i和j，且i在j之前进入流水线，

可能发生的数据冲突有：

- **写后读冲突（RAW）**

在 i 写入之前，j 先去读。
j 读出的内容是错误的。

这是最常见的一种数据冲突，它对应于真数据相关。

- **写后写冲突（WAW）**

  在 i 写入之前，j 先写。

  最后写入的结果是 i 的。错误!

  这种冲突对应于输出相关。

写后写冲突仅发生在这样的流水线中：

- 流水线中不只一个段可以进行写操作；
- 指令被重新排序了。

前面介绍的5段流水线不会发生写后写冲突。

（只在WB段写寄存器）

- **读后写冲突（WAR）**

  在 i 读之前，j 先写。

  i 读出的内容是错误的！

  由反相关引起。

这种冲突仅发生在这样的情况下：

- 有些指令的写结果操作提前了，而且有些指令的读操作滞后了；
- 指令被重新排序了。

2.3 指令的动态调度

➢ 静态调度

- 依靠编译器对代码进行静态调度，以减少相关和冲突。
- 它不是在程序执行的过程中、而是在**编译期间**进行代码调度和优化。
- 通过把相关的指令拉开距离来减少可能产生的停顿。

2.3 指令的动态调度

➢ 概念

- 在程序的**执行过程**中，依靠专门硬件对代码进行调度，减少数据相关导致的停顿。

➢ 动态调度的优点：

- ❑ 能够处理一些在编译时情况不明的相关（比如涉及到存储器访问的相关），并简化了编译器；
- ❑ 能够使本来是面向某一流水线优化编译的代码在其它的流水线（动态调度）上也能高效地执行。

➢ 以硬件复杂性的显著增加为代价

1. 到目前为止我们所使用流水线的最大的局限性:

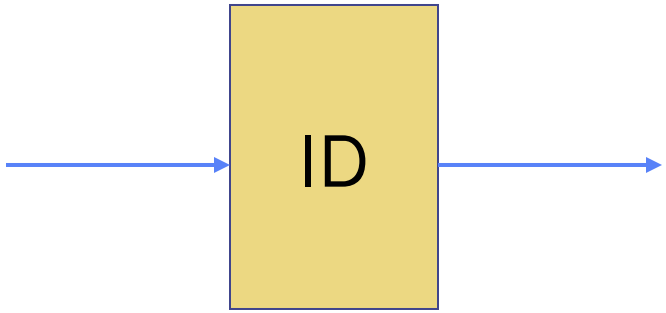
- 指令是按序流出和按序执行的
- 考虑下面一段代码:

```
DIV.D     F4，F0，F2
ADD.D     F10，F4，F6
SUB.D     F12，F6，F14
```

ADD.D指令与DIV.D指令关于F4相关，导致流水线停顿。

SUB.D指令与流水线中的任何指令都没有关系，但也因此受阻。

在前面的基本流水线中：



检测结构冲突

检测数据冲突

一旦一条指令受阻，其后的指令都将停顿。

- 为了使上述指令序列中的SUB.D指令能继续执行下去，必须把指令流出的工作拆分为两步：
  - 检测结构冲突
  - 等待数据冲突消失

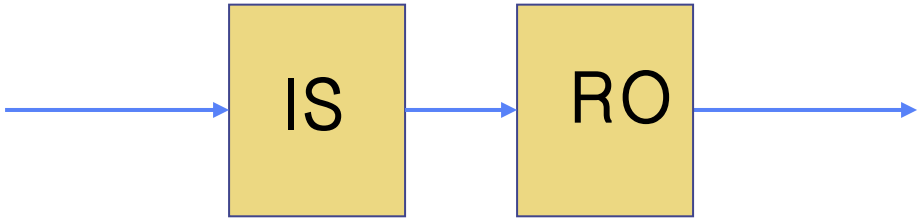
  只要检测到没有结构冲突，就可以让指令流出。并且流出后的指令一旦其操作数就绪就可以立即执行。

乱序执行

- 指令的执行顺序与程序顺序不相同
- 指令的完成也是乱序完成的
  - 即指令的完成顺序与程序顺序不相同。

2.为了支持乱序执行，我们将5段流水线的译码阶段再分为两个阶段：

- 流出（Issue，IS）：指令译码，检查是否存在结构冲突。 （in-order issue)
- 读操作数（Read Operands，RO）：等待数据冲突消失，然后读操作数。 (out of order execution)



检测结构冲突　　检测数据冲突

3. 在前述5段流水线中，是不会发生WAR冲突和WAW冲突的。但乱序执行就使得它们可能发生了。

- 例如，考虑下面的代码

```
DIV.D   F10, F0, F2
ADD.D   F10, F4, F6
SUB.D   F6, F8, F14
```

存在输出相关：DIV.D 与 ADD.D（F10）

存在反相关：ADD.D 与 SUB.D（F6）

可以通过使用寄存器重命名来消除。

4. 动态调度的流水线支持多条指令同时处于执行当中。

- 要求：具有多个功能部件、或者功能部件流水化、或者兼而有之。
- 我们假设具有多个功能部件。

5. 指令乱序完成带来的最大问题：

异常处理比较复杂

(精确异常处理、不精确异常处理)

- 动态调度的处理机要保持正确的异常行为
  - 对于一条会产生异常的指令来说，只有当处理机确切地知道该指令将被执行时，才允许它产生异常。

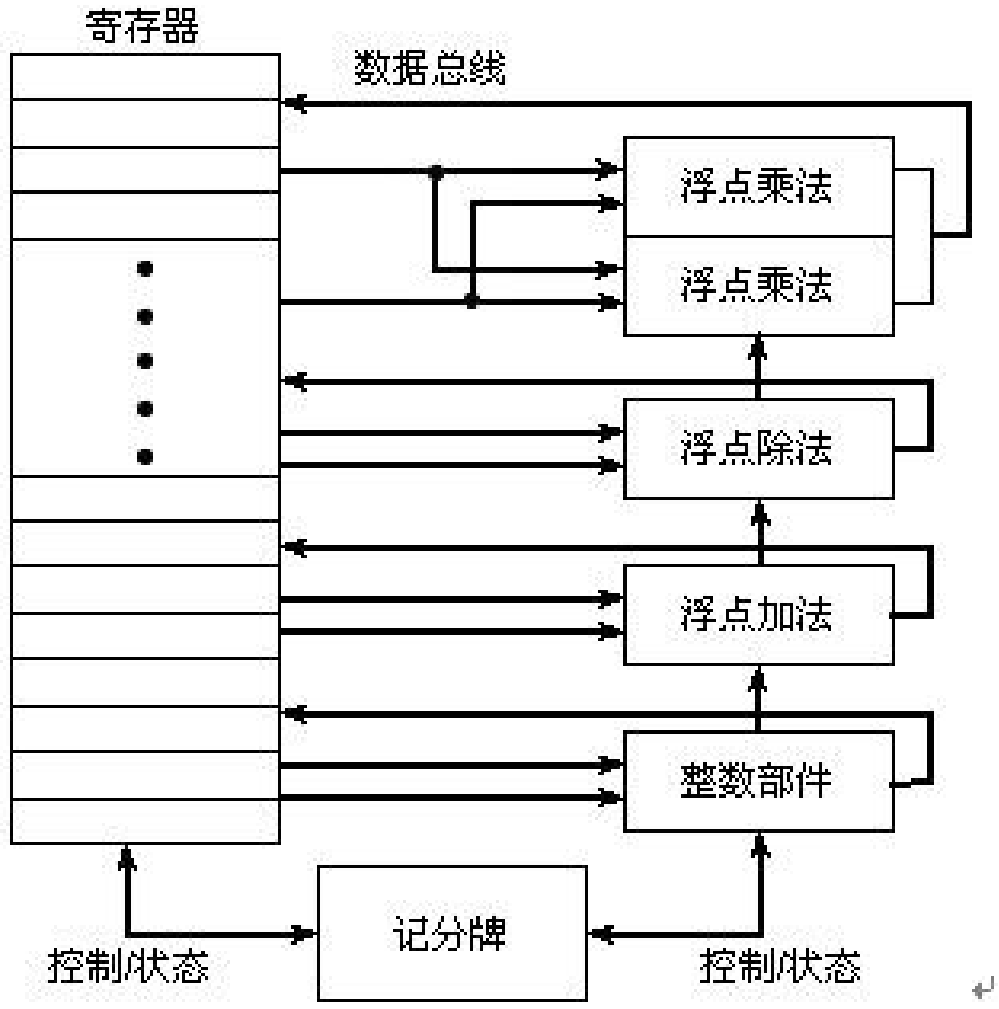
2、硬件方法-Scoreboard

1. 基本思想

- CDC 6600计算机最早采用此功能
  - 该机器用一个称为记分牌的硬件实现了对指令的动态调度。
  - 该硬件中维护着3张表，分别用于记录**指令的执行状态**、**功能部件状态**、**寄存器状态**以及**数据相关关系**等。
  - 它把前述5段流水线中的译码段ID分解成了两个段：流出和读操作数，以避免当某条指令在ID段被停顿时挡住后面无关指令的流动。

- 记分牌的目标：在没有结构冲突时，尽可能早地执行没有数据冲突的指令，实现每个时钟周期执行一条指令。
- 要发挥指令乱序执行的好处，必须有多条指令同时处于执行阶段。
  - CDC 6600具有16个独立的功能部件
    - 4个浮点部件
    - 5个访存部件
    - 7个整数操作部件
- 假设
  - 所考虑的处理器有2个乘法器、1个加法器、1个除法部件和1个整数部件。
  - 整数部件用来处理所有的存储器访问、分支处理和整数操作。

- 采用了记分牌的MIPS处理器的基本结构
  - 每条指令都要经过记分牌。
  - 记分牌负责相关检测并控制指令的流出和执行。



- 每条指令的执行过程分为4段 （主要看浮点操作数，因而不考虑涉及访问内存的阶段）
  - 流出

    如果当前流出指令所需的功能部件空闲，并且所有其他正在执行的指令的目的寄存器与该指令的不同，记分牌就向功能部件流出该指令，并修改记分牌内部的记录表。

    解决了结构冲突与WAW冲突

- **读操作数**

记分牌监测源操作数的可用性，如果数据可用，它就通知功能部件从寄存器中读出源操作数并开始执行。

动态地解决了RAW冲突，并导致指令可能乱序开始执行。

- **执行**

取到操作数后，功能部件开始执行。当产生出结果后，就通知记分牌它已经完成执行。

在浮点流水线中，这一段可能要占用多个时钟周期。

- 写结果

  记分牌一旦知道执行部件完成了执行，就检测是否存在WAR冲突。如果不存在，或者原有的WAR冲突已消失，记分牌就通知功能部件把结果写入目的寄存器，并释放该指令使用的所有资源。

➢ 如果检测到WAR冲突，就不允许该指令将结果写到目的寄存器。这发生在以下情况：

- 前面的某条指令（按顺序流出）还没有读取操作数；而且：其中某个源操作数寄存器与本指令的目的寄存器相同。
- 在这种情况下，记分牌必须等待，直到该冲突消失。

- 记分牌中记录的信息由3部分构成
  - 指令状态表：记录正在执行的各条指令已经进入到了哪一段。
  - 功能部件状态表：记录各个功能部件的状态。每个功能部件有一项，每一项由以下9个字段组成：
    - **Busy**：忙标志，指出功能部件是否忙。初值为“no”；
    - **Op**：该功能部件正在执行或将要执行的操作；
    - **Fi**：目的寄存器编号；
    - **Fj，Fk**：源寄存器编号；
    - **Qj，Qk**：指出向源寄存器Fj、Fk写数据的功能部件；
    - **Rj，Rk**：标志位，为“yes”表示Fj，Fk中的操作数就绪且还未被取走。否则就被置为“no”。

- 结果寄存器状态表Result：每个寄存器在该表中有一项，用于指出哪个功能部件（编号）将把结果写入该寄存器。
  - 如果当前正在运行的指令都不以它为目的寄存器，则其相应项置为“no”。
  - Result各项的初值为“no”（全0）。

```
L.D      F6, 34(R2)
L.D      F2, 45(R3)
MULT.D   F0, F2, F4
SUB.D    F8, F6, F2
DIV.D    F10, F0, F6
ADD.D    F6, F8, F2
```

例2.1 假设浮点流水线中各部件的延迟如下：

加法需2个时钟周期

乘法需10个时钟周期

除法需40个时钟周期

代码段和记分牌信息的起始点状态如上图。分别给出MULT.D和DIV.D准备写结果之前的记分牌状态。

解 图中的代码段存在以下相关性：

(1) 先写后读相关：第二条**L.D**指令到**MULT.D**和**SUB.D**之间，

MULT.D到**DIV.D**之间，**SUB.D**到**ADD.D**之间；

(2) 先读后写相关：**DIV.D**和**ADD.D**之间，**SUB.D**和**ADD.D**之间；

(3) 结构相关：**ADD.D**和**SUB.D**指令关于浮点加法部件。

Cycle 0

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)				
L.D F2, 45(R3)				
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer									
Mult1									
Mult2									
Add									
Divide									

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件								

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 1

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1			
L.D	F2, 45(R3)				
MULT.D	F0, F2, F4				
SUB.D	F8, F6, F2				
DIV.D	F10, F0, F6				
ADD.D	F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	yes	L.D	F6		R2				yes
Mult1									
Mult2									
Add									
Divide									

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件				Integer				

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 2

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2		
L.D F2, 45(R3)				
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	yes	L.D	F6		R2				no
Mult1									
Mult2									
Add									
Divide									

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件				Integer				

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 3

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	
L.D	F2, 45(R3)				
MULT.D	F0, F2, F4				
SUB.D	F8, F6, F2				
DIV.D	F10, F0, F6				
ADD.D	F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	yes	L.D	F6		R2				no
Mult1									
Mult2									
Add									
Divide									

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件				Integer				

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 4

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)				
MULT.D	F0, F2, F4				
SUB.D	F8, F6, F2				
DIV.D	F10, F0, F6				
ADD.D	F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no	~~L.D~~	~~F6~~		~~R2~~				~~no~~
Mult1									
Mult2									
Add									
Divide									

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件				~~Integer~~				

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Cycle 5

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5			
MULT.D	F0, F2, F4				
SUB.D	F8, F6, F2				
DIV.D	F10, F0, F6				
ADD.D	F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	yes	L.D	F2		R3				yes
Mult1									
Mult2									
Add									
Divide									

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件		Integer						

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 6

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6		
MULT.D F0, F2, F4	6			
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	yes	L.D	F2		R3				no
Mult1	yes	MULT.D	F0	F2	F4	Integer		no	yes
Mult2									
Add									
Divide									

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1	Integer						

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 7

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	
MULT.D	F0, F2, F4	6			
SUB.D	F8, F6, F2	7			
DIV.D	F10, F0, F6				
ADD.D	F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	yes	L.D	F2		R3				no
Mult1	yes	MULT.D	F0	F2	F4	Integer		no	yes
Mult2									
Add	yes	SUB.D	F8	F6	F2		Integer	yes	no

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1	Integer			Add			

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Cycle 8

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6			
SUB.D	F8, F6, F2	7			
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no	~~L.D~~	~~F2~~		~~R3~~				~~no~~
Mult1	yes	MULT.D	F0	F2	F4	~~Integer~~		yes	yes
Mult2									
Add	yes	SUB.D	F8	F6	F2		~~Integer~~	yes	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1	~~Integer~~			Add	Divide		

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Cycle 9

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9		
SUB.D F8, F6, F2	7	9		
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	SUB.D	F8	F6	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1				Add	Divide		

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 10

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	10	
SUB.D	F8, F6, F2	7	9	10	
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	SUB.D	F8	F6	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1				Add	Divide		

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Cycle 11

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	11	
SUB.D	F8, F6, F2	7	9	11	
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	SUB.D	F8	F6	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1				Add	Divide		

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Cycle 12

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	12	
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2				

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	no	~~SUB.D~~	~~F8~~	~~F6~~	~~F2~~			~~no~~	~~no~~
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1				~~Add~~	Divide		

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 13

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	13	
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2	13			

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	ADD.D	F6	F8	F2			yes	yes
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1			Add		Divide		

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Cycle 14

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	14	
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2	13	14		

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	ADD.D	F6	F8	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1			Add		Divide		

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Cycle 15

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	15	
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2	13	14	15	

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	ADD.D	F6	F8	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1			Add		Divide		

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 16

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	16	
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	ADD.D	F6	F8	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1			Add		Divide		

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Cycle 17

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9	17	
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	ADD.D	F6	F8	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1			Add		Divide		

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

注意：**ADD.D在该周期无法WB（WAR冲突）**

Cycle 18

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	18	
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	ADD.D	F6	F8	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1			Add		Divide		

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

注意：**ADD.D在该周期无法WB（WAR冲突）**

Cycle 19

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9	19	
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8			
ADD.D F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2									
Add	yes	ADD.D	F6	F8	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Multi1			Add		Divide		

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

注意：**ADD.D在该周期无法WB（WAR冲突）**

Cycle 20

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	19	20
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8			
ADD.D	F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	no	~~MULT.D~~	~~F0~~	~~F2~~	~~F4~~			~~no~~	~~no~~
Mult2									
Add	yes	ADD.D	F6	F8	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	~~Mult1~~		yes	yes

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	~~Multi1~~			Add		Divide		

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

注意：**ADD.D在该周期无法WB（WAR冲突）**

Cycle 21

指令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MULT.D F0, F2, F4	6	9	19	20
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8	21		
ADD.D F6, F8, F2	13	14	16	

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	no								
Mult2									
Add	yes	ADD.D	F6	F8	F2			no	no
Divide	yes	DIV.D	F10	F0	F6			no	no

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件				Add		Divide		

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

注意：**ADD.D在该周期无法WB（WAR冲突）**

Cycle 22

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	19	20
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8	21	22	
ADD.D	F6, F8, F2	13	14	16	22

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	no								
Mult2									
Add	no	~~ADD.D~~	~~F6~~	~~F8~~	~~F2~~			~~no~~	~~no~~
Divide	yes	DIV.D	F10	F0	F6			no	no

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件				~~Add~~		Divide		

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

注意：**ADD.D可以WB**

Cycle 23

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	19	20
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8	21	23	
ADD.D	F6, F8, F2	13	14	16	22

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	no								
Mult2									
Add	no								
Divide	yes	DIV.D	F10	F0	F6			no	no

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件						Divide		

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Cycle 61

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	19	20
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8	21	61	
ADD.D	F6, F8, F2	13	14	16	22

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	no								
Mult2									
Add	no								
Divide	yes	DIV.D	F10	F0	F6			no	no

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件						Divide		

L.D F6, 34(R2)
L.D F2, 45(R3)
MULT.D F0, F2, F4
SUB.D F8, F6, F2
DIV.D F10, F0, F6
ADD.D F6, F8, F2

Cycle 62

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	2	3	4
L.D	F2, 45(R3)	5	6	7	8
MULT.D	F0, F2, F4	6	9	19	20
SUB.D	F8, F6, F2	7	9	11	12
DIV.D	F10, F0, F6	8	21	61	62
ADD.D	F6, F8, F2	13	14	16	22

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	no								
Mult2									
Add	no								
Divide	~~no~~	~~DIV.D~~	~~F10~~	~~F0~~	~~F6~~			~~no~~	~~no~~

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件						~~Divide~~		

L.D	F6, 34(R2)
L.D	F2, 45(R3)
MULT.D	F0, F2, F4
SUB.D	F8, F6, F2
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

指　　令	指令状态表			
	流出	读操作数	执行	写结果
L.D F6, 34(R2)	√	√	√	√
L.D F2, 45(R3)	√	√	√	√
MULT.D F0, F2, F4	√	√	√	
SUB.D F8, F6, F2	√	√	√	√
DIV.D F10, F0, F6	√			
ADD.D F6, F8, F2	√	√	√	

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MULT.D	F0	F2	F4			no	no
Mult2	no								
Add	yes	ADD.D	F6	F8	F2			no	no
Divide	yes	DIV.D	F10	F0	F6	Mult1		no	yes

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件名称	Mult1			Add		Divide		

程序段执行到MULT.D将要写结果时记分牌的状态

指　令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	√	√	√	√
L.D	F2, 45(R3)	√	√	√	√
MULT.D	F0, F2, F4	√	√	√	√
SUB.D	F8, F6, F2	√	√	√	√
DIV.D	F10, F0, F6	√	√	√	
ADD.D	F6, F8, F2	√	√	√	√

部件名称	功能部件状态表								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	no								
Mult2	no								
Add	no								
Divide	yes	DIV.D	F10	F0	F6			no	no

	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件名称						Divide		

程序段执行到DIV.D将要写结果时记分牌的状态

1. 具体算法

约定：

- **FU**：表示当前指令所要用的功能部件；
- **D**：目的寄存器的名称；
- **S1、S2**：源操作数寄存器的名称；
- **Op**：要进行的操作；
- **Fj[FU]**：功能部件FU的Fj字段（其他字段依此类推）；
- **Result[D]**：结果寄存器状态表中与寄存器D相对应的内容。其中存放的是将把结果写入寄存器D的功能部件的名称。

（1）指令流出

进入条件：

not Busy[FU] & not Result[D]； // 功能部件空闲且没有
//写后写（WAW）冲突。

计分牌内容修改：

Busy[FU]←yes； // 把当前指令的相关信息填入功能部件状态表。

Op[FU]←Op； // 记录操作码。

Fi[FU]←D； // 记录目的寄存器编号。

Fj[FU]←S1； // 记录第一个源寄存器编号。

Fk[FU]←S2；　　// 记录第二个源寄存器编号。

Qj[FU]←Result[S1]；　// 记录将产生第一个源操作数的部件。

Qk[FU]←Result[S2]；　// 记录将产生第二个源操作数的部件。

Rj[FU]←not Qj[FU]；　// 置第一个源操作数是否可用的标志。

如果Qj[FU]为“no”，就表示没有操作部件要写S1，数据可用。

置Rj[FU]为“yes”。否则置Rj[FU]为“no”。

Rk[FU]←not Qk[FU]；　// 置第二个源操作数是否可用的标志。

Result[D]←FU；　　// D是当前指令的目的寄存器。功能

部件FU将把结果写入D。

(2) 读操作数

进入条件：

Rj[FU] & Rk[FU]； // 两个源操作数都已就绪。

计分牌内容修改：

Rj[FU]←no； // 已经读走了就绪的第一个源操作数。

Rk[FU]←no； // 已经读走了就绪的第二个源操作数。

Qj[FU]←0； // 不再等待其他FU的计算结果。

Qk[FU]←0；

(3) 执行

结束条件：

功能部件操作结束。

(4) 写结果

进入条件：

∀f((Fj[f]≠Fi[FU] or Rj[f]=no)

& (Fk[f]≠Fi[FU] or Rk[f]=no))； // 不存在WAR冲突。

计分牌内容修改：

∀f(if Qj[f]=FU then Rj[f]←yes)；　　// 如果有指令在等待该结果（作为第一源操作数），则将其Rj置为“yes”，表示数据可用。

∀f(if Qk[f]=FU then Rk[f]←yes)；　　// 如果有指令在等待该结果（作为第二源操作数），则将其Rk置为“yes”，表示数据可用。

Result(Fi[FU])←0；　　// 释放目的寄存器Fi[FU]。

Busy[FU]=no；　　// 释放功能部件FU。

- 记分牌的性能受限于以下几个方面：
  - 程序代码中可开发的并行性，即是否存在可以并行执行的不相关的指令。
  - 记分牌的容量。
    - 记分牌的容量决定了流水线能在多大范围内寻找不相关指令。流水线中可以同时容纳的指令数量称为指令窗口。
  - 功能部件的数目和种类。
    - 功能部件的总数决定了结构冲突的严重程度。
  - 反相关和输出相关。
    - 它们引起计分牌中更多的WAR和WAW冲突 。

2、硬件方法-Tomasulo

1. 核心思想
   - 记录和检测指令相关，操作数一旦就绪就立即执行，把发生RAW冲突的可能性减少到最小；
   - 通过寄存器换名来消除WAR冲突和WAW冲突。
2. IBM 360/91首先采用了Tomasulo算法。
   - IBM 360/91的设计目标是基于整个360系列的统一指令系统和编译器来实现高性能，而不是设计和利用专用的编译器来提高性能。

     需要更多地依赖于硬件。

- IBM 360体系结构只有4个双精度浮点寄存器，限制了编译器调度的有效性。
- 360/91的访存时间和浮点计算时间都很长。

  (也是Tomasulo算法要解决的问题)

1. 寄存器换名可以消除WAR冲突和WAW冲突。

- 考虑以下代码：

```
DIV.D   F0, F2, F4
ADD.D   F6, F0, F8
S.D             F6, 0 (R1)
SUB.D   F8, F10, F14
MUL.D   F6, F10, F8
```

反相关（F8）导致WAR冲突

输出相关（F6）导致WAW冲突

消除名相关

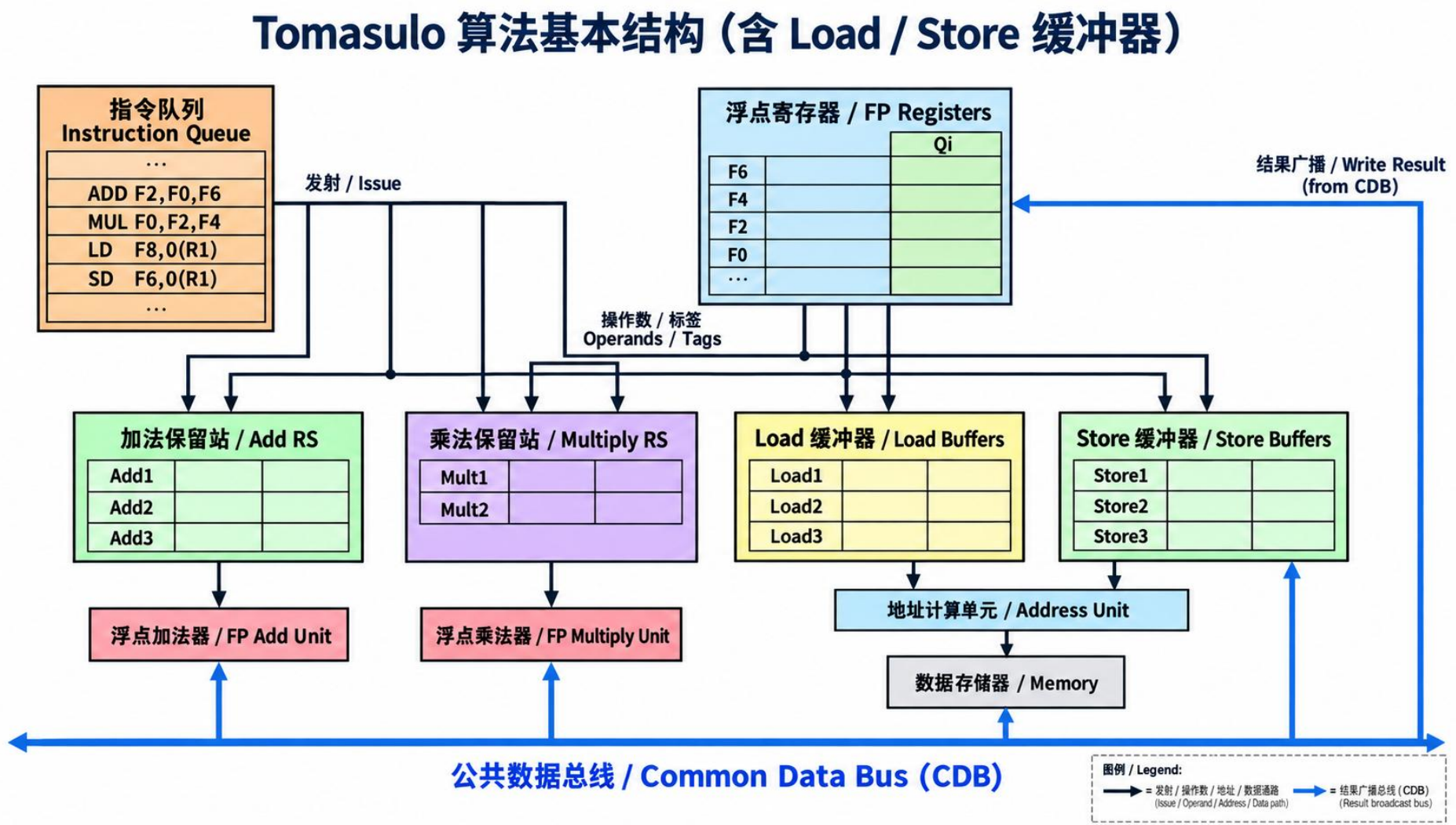
- 引入两个临时寄存器S和T
- 把这段代码改写为：

```
DIV.D   F0, F2, F4
ADD.D   S, F0, F8
S.D             S, 0 (R1)
SUB.D   T, F10, F14
MUL.D   F6, F10, T
```

两个F6都换名为S

两个F8都换名为T

1. 基于Tomasulo算法的MIPS处理器浮点部件的基本结构



➢ 保留站（reservation station）

每个保留站中保存一条已经流出并等待到本功能部件执行的指令（相关信息）。

包括：操作码、操作数以及用于检测和解决冲突的信息。

- 在一条指令流出到保留站的时候，如果该指令的源操作数已经在寄存器中就绪，则将之取到该保留站中。
- 如果操作数还没有计算出来，则在该保留站中记录将产生这个操作数的保留站的标识。

- 浮点加法器有3个保留站：ADD1，ADD2，ADD3
- 浮点乘法器有两个保留站：MULT1，MULT2
- 每个保留站都有一个标识字段，唯一地标识了该保留站。

- 公共数据总线CDB

  (一条重要的数据通路)

  - 所有功能部件的计算结果都是送到CDB上，由它把这些结果直接送到（播送到）各个需要该结果的地方。
  - 在具有多个执行部件且采用多流出（即每个时钟周期流出多条指令）的流水线中，需要采用多条CDB。

load缓冲器和store缓冲器

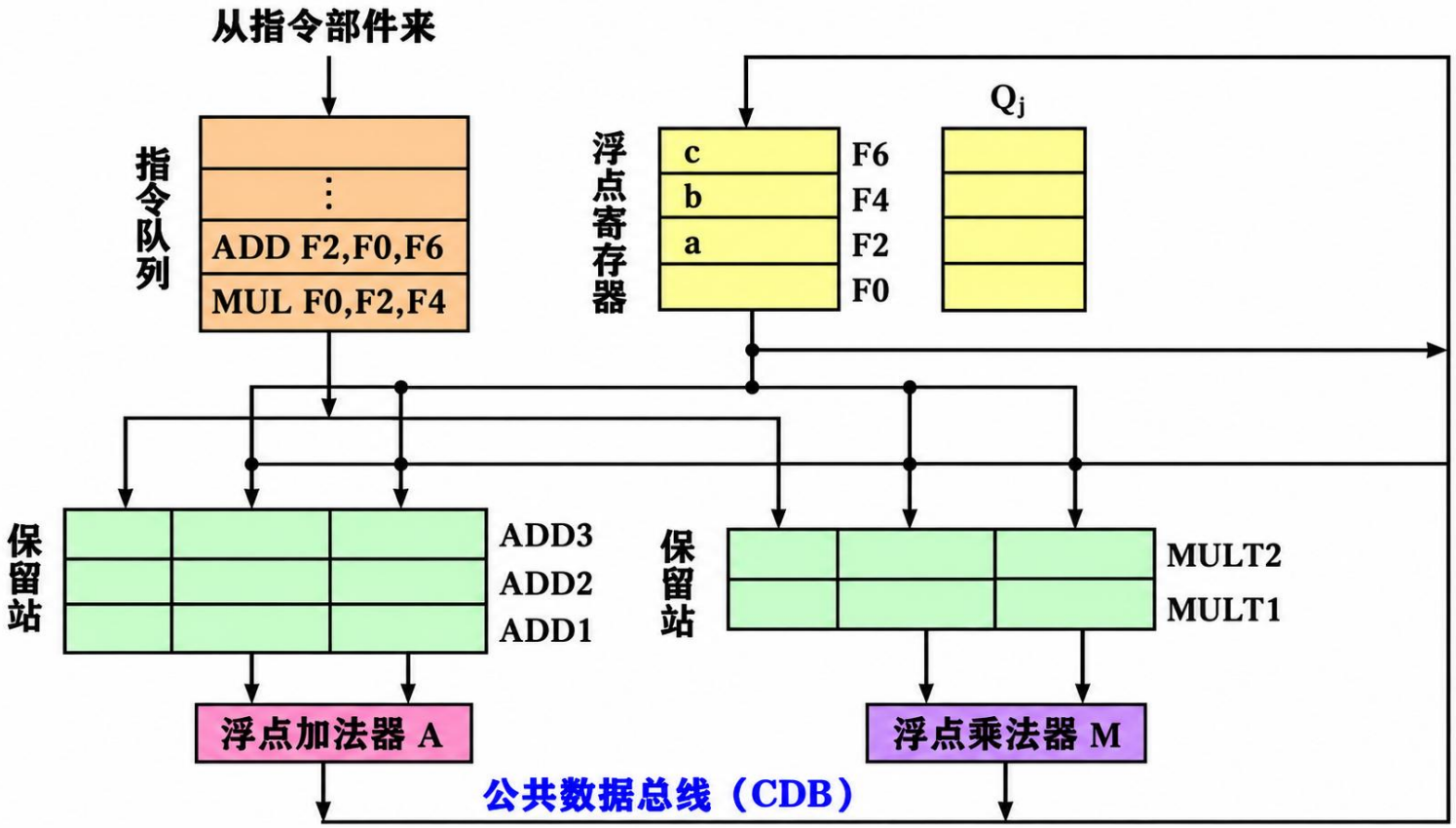
- 存放读/写存储器的数据或地址
- load缓冲器的作用有3个：
  - 存放用于计算有效地址的分量；
  - 记录正在进行的load访存，等待存储器的响应；
  - 保存已经完成了的load的结果（即从存储器取来的数据），等待CDB传输。
- store缓冲器的作用有3个：
  - 存放用于计算有效地址的分量；
  - 保存正在进行的store访存的目标地址，该store正在等待存储数据的到达；
  - 保存该store的地址和数据，直到存储部件接收。

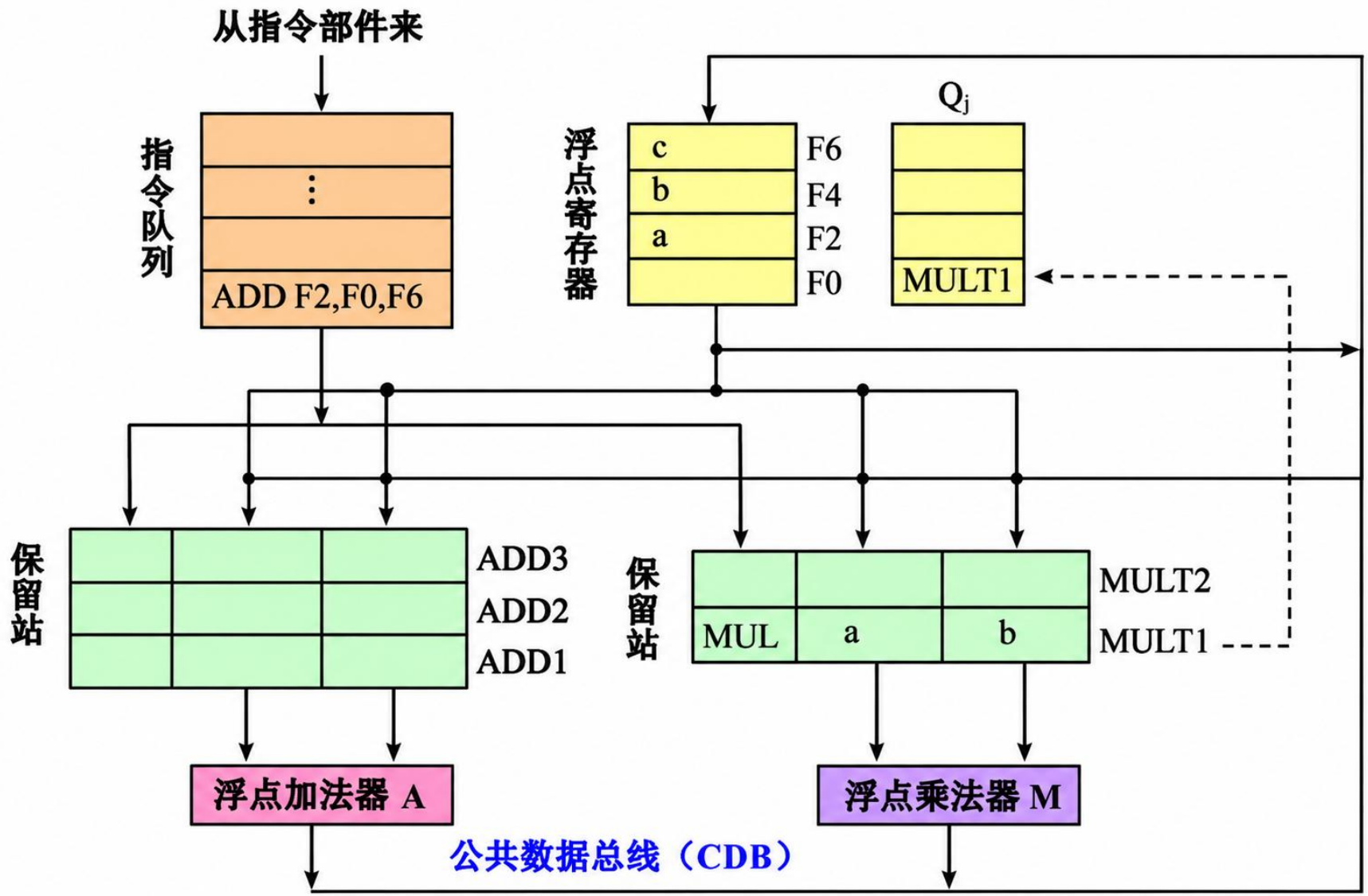
- 浮点寄存器FP
  - 共有16个浮点寄存器：F0，F2，F4，…，F30。
  - 它们通过一对总线连接到功能部件，并通过CDB连接到store缓冲器。
- 指令队列
  - 指令部件送来的指令放入指令队列
  - 指令队列中的指令按先进先出的顺序流出
- 运算部件
  - 浮点加法器完成加法和减法操作
  - 浮点乘法器完成乘法和除法操作

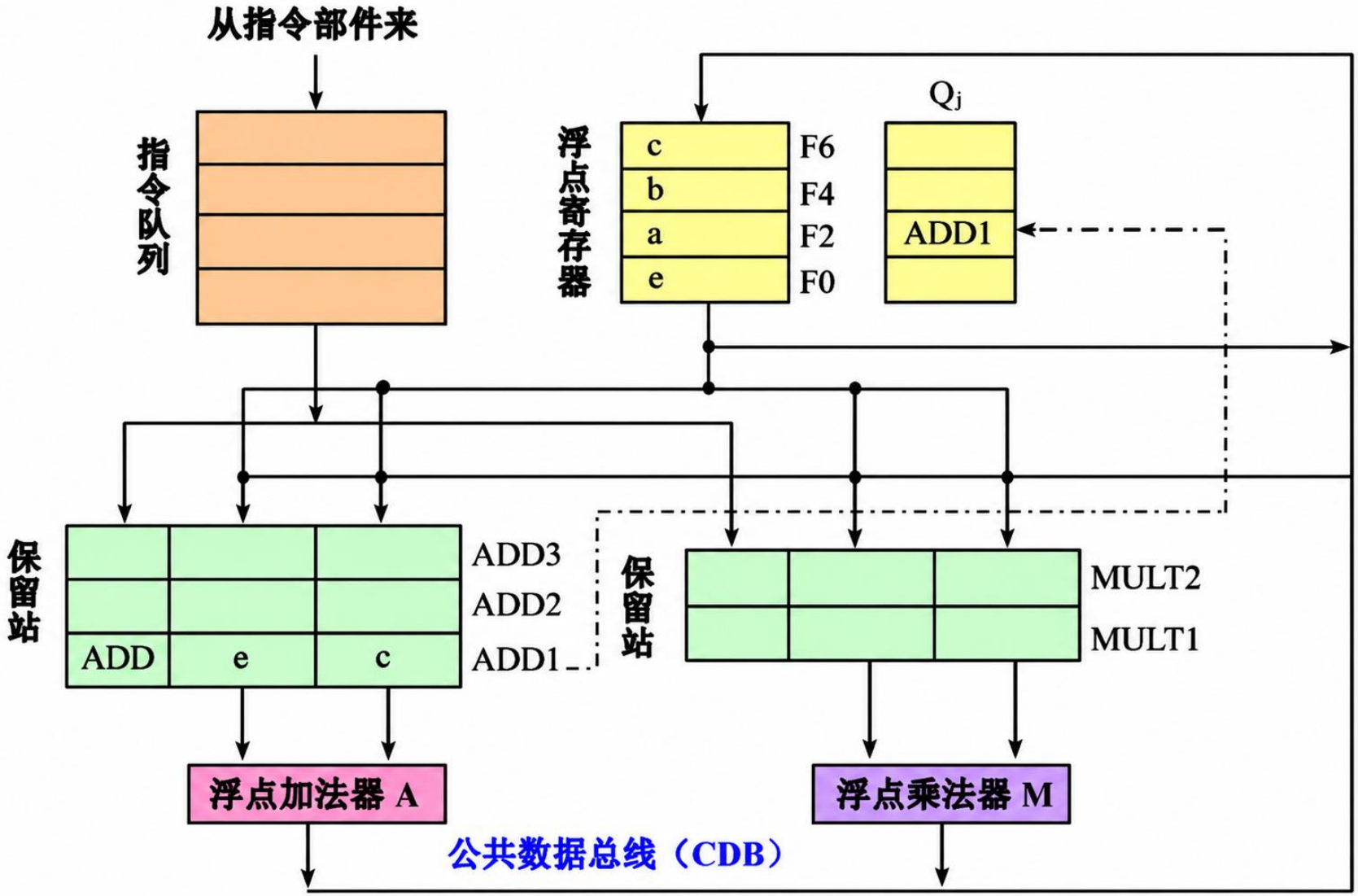
2. 在Tomasulo算法中，寄存器换名是通过保留站和流出逻辑来共同完成的。

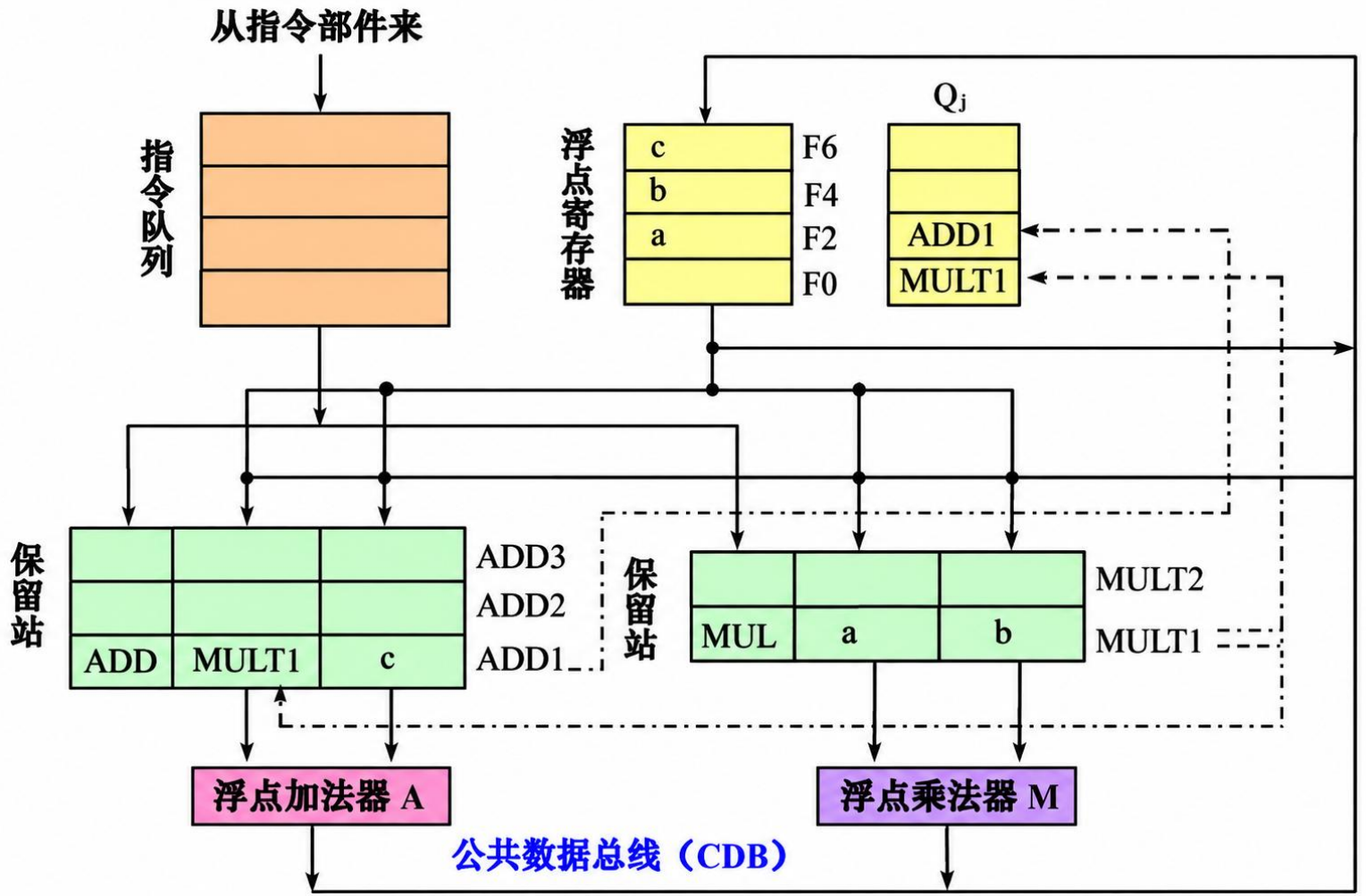
- 当指令流出时，如果其操作数还没有计算出来，则将该指令中相应的寄存器号换名为将产生这个操作数的保留站的标识。
- 指令流出到保留站后，其操作数寄存器号或者换成了数据本身（如果该数据已经就绪），或者换成了保留站的标识，不再与寄存器有关系。

3. 通过一个简单的例子来说明Tomasulo算法的基本思想




从指令部件来
指令队列
ADD F2,F0,F6
浮点寄存器
c
b
a
F6
F4
F2
F0
Qj
MULT1
保留站
ADD3
ADD2
ADD1
保留站
MUL
a
b
MULT2
MULT1
浮点加法器 A
浮点乘法器 M
公共数据总线（CDB）


从指令部件来
指令队列
浮点寄存器
c
b
a
e
F6
F4
F2
F0
Qj
ADD1
保留站
ADD3
ADD2
ADD1
ADD
e
c
保留站
MULT2
MULT1
浮点加法器 A
浮点乘法器 M
公共数据总线（CDB）


从指令部件来
指令队列
浮点寄存器
c
b
a
F6
F4
F2
F0
Qj
ADD1
MULT1
保留站
ADD3
ADD2
ADD1
ADD
MULT1
c
保留站
MULT2
MULT1
MUL
a
b
浮点加法器 A
浮点乘法器 M
公共数据总线（CDB）

4. Tomasulo算法具有以下两个特点：

- 冲突检测和指令执行控制是分布的。
  - 每个功能部件的保留站中的信息决定了什么时候指令可以在该功能部件开始执行。

- 计算结果通过CDB直接从产生它的保留站传送到所有需要它的功能部件，而不用经过寄存器。

5. 指令执行的步骤

使用Tomasulo算法的流水线需3段：

- 流出：从指令队列的头部取一条指令。
  - 如果该指令的操作所要求的保留站有空闲的，就把该指令送到该保留站（设为r）。
    - 如果其操作数在寄存器中已经就绪，就将这些操作数送入保留站r。
    - 如果其操作数还没有就绪，就把将产生该操作数的保留站的标识送入保留站r。
    - 一旦被记录的保留站完成计算，它将直接把数据送给保留站r。

    （寄存器换名和对操作数进行缓冲，消除WAR冲突）

- 完成对目标寄存器的预约工作

  （消除了WAW冲突）

- 如果没有空闲的保留站，指令就不能流出

  （发生了结构冲突）

执行

- 当两个操作数都就绪后，本保留站就用相应的功能部件开始执行指令规定的操作。
- load和store指令的执行需要两个步骤:
  - 计算有效地址（要等到基地址寄存器就绪）
  - 把有效地址放入load或store缓冲器

写结果

- 功能部件计算完毕后，就将计算结果放到CDB上，所有等待该计算结果的寄存器和保留站（包括store缓冲器）都同时从CDB上获得所需要的数据。

6. 每个保留站有以下7个字段：

- Op：要对源操作数进行的操作
- Qj，Qk：将产生源操作数的保留站号
  - 等于0表示操作数已经就绪且在Vj或Vk中，或者不需要操作数。
- Vj，Vk：源操作数的值
  - 对于每一个操作数来说，V或Q字段只有一个有效。
  - 对于load来说，Vk字段用于保存偏移量。
- Busy：为“yes”表示本保留站或缓冲单元“忙”
- A：仅load和store缓冲器有该字段。开始是存放指令中的立即数字段，地址计算后存放有效地址。

- Qi：寄存器状态表
  - 每个寄存器在该表中有对应的一项，用于存放将把结果写入该寄存器的保留站的站号。
  - 为0表示当前没有正在执行的指令要写入该寄存器，也即该寄存器中的内容就绪。

例2.2 对于下述指令序列，给出当第一条指令完成并写入结果时，Tomasulo算法所用的各信息表中的内容。

```
L.D    F6,34  (R2)
L.D    F2,45  (R3)
MUL.D        F0,F2,F4
SUB.D        F8,F2,F6
DIV.D        F10,F0,F6
ADD.D        F6,F8,F2
```

Cycle 0

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)			
L.D F2, 45(R3)			
MULT.D F0, F2, F4			
SUB.D F8, F6, F2			
DIV.D F10, F0, F6			
ADD.D F6, F8, F2			

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1						
Add2						
Add3						
Mul1						
Mul2						

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件								

Cycle 1

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1		
L.D F2, 45(R3)			
MULT.D F0, F2, F4			
SUB.D F8, F6, F2			
DIV.D F10, F0, F6			
ADD.D F6, F8, F2			

	Load保留站	
	Busy	Address
Load1	yes	34+R2
Load2		
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1						
Add2						
Add3						
Mul1						
Mul2						

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件				Load1				

Cycle 2

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2	
L.D F2, 45(R3)	2		
MULT.D F0, F2, F4			
SUB.D F8, F6, F2			
DIV.D F10, F0, F6			
ADD.D F6, F8, F2			

	Load保留站	
	Busy	Address
Load1	yes	34+R2
Load2	yes	45+R3
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1						
Add2						
Add3						
Mul1						
Mul2						

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件		Load2		Load1				

Cycle 3

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	
L.D F2, 45(R3)	2	3	
MULT.D F0, F2, F4	3		
SUB.D F8, F6, F2			
DIV.D F10, F0, F6			
ADD.D F6, F8, F2			

	Load保留站	
	Busy	Address
Load1	yes	34+R2
Load2	yes	45+R3
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1						
Add2						
Add3						
Mul1	yes	MULT.D		Value(F4)	Load2	
Mul2						

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	Load2		Load1				

Cycle 4

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	
MULT.D F0, F2, F4	3		
SUB.D F8, F6, F2	4		
DIV.D F10, F0, F6			
ADD.D F6, F8, F2			

	Load保留站	
	Busy	Address
Load1	no	~~34+R2~~
Load2	yes	45+R3
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	yes	SUB.D	M1			Load2
Add2						
Add3						
Mul1	yes	MULT.D		Value(F4)	Load2	
Mul2						

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	Load2		M1	Add1			

Cycle 5

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3		
SUB.D F8, F6, F2	4		
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2			

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	~~45+R3~~
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	yes	SUB.D	M1	M2		~~Load2~~
Add2						
Add3						
Mul1	yes	MULT.D	M2	Value(F4)	~~Load2~~	
Mul2	yes	DIV.D		M1	Mul1	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	M2		M1	Add1	Mul2		

Cycle 6

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6	
SUB.D F8, F6, F2	4	6	
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6		

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	yes	SUB.D	M1	M2		
Add2	yes	ADD.D		M2	Add1	
Add3						
Mul1	yes	MULT.D	M2	Value(F4)		
Mul2	yes	DIV.D		M1	Mul1	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	M2		Add2	Add1	Mul2		

Cycle 7

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6-7	
SUB.D F8, F6, F2	4	6-7	
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6		

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	yes	SUB.D	M1	M2		
Add2	yes	ADD.D		M2	Add1	
Add3						
Mul1	yes	MULT.D	M2	Value(F4)		
Mul2	yes	DIV.D		M1	Mul1	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	M2		Add2	Add1	Mul2		

Cycle 8

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6-8	
SUB.D F8, F6, F2	4	6-7	8
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6		

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	no	~~SUB.D~~	~~M1~~	~~M2~~		
Add2	yes	ADD.D	Val1	M2	~~Add1~~	
Add3						
Mul1	yes	MULT.D	M2	Value(F4)		
Mul2	yes	DIV.D		M1	Mul1	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	M2		Add2	Val1	Mul2		

Cycle 9

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6-9	
SUB.D F8, F6, F2	4	6-7	8
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9	

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	no					
Add2	yes	ADD.D	Val1	M2		
Add3						
Mul1	yes	MULT.D	M2	Value(F4)		
Mul2	yes	DIV.D		M1	Mul1	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	M2		Add2	Val1	Mul2		

Cycle 10

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6-10	
SUB.D F8, F6, F2	4	6-7	8
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9-10	

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	no					
Add2	yes	ADD.D	Val1	M2		
Add3						
Mul1	yes	MULT.D	M2	Value(F4)		
Mul2	yes	DIV.D		M1	Mul1	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	M2		Add2	Val1	Mul2		

Cycle 11

指令		指令状态表		
		流出	执行	写结果
L.D	F6, 34(R2)	1	2,3	4
L.D	F2, 45(R3)	2	3-4	5
MULT.D	F0, F2, F4	3	6-11	
SUB.D	F8, F6, F2	4	6-7	8
DIV.D	F10, F0, F6	5		
ADD.D	F6, F8, F2	6	9-10	11

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	no					
Add2	no	~~ADD.D~~	~~Val1~~	~~M2~~		
Add3						
Mul1	yes	MULT.D	M2	Value(F4)		
Mul2	yes	DIV.D		M1	Mul1	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	M2		Val2	Val1	Mul2		

Cycle 15

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6-15	
SUB.D F8, F6, F2	4	6-7	8
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9-10	11

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	no					
Add2	no					
Add3						
Mul1	yes	MULT.D	M2	Value(F4)		
Mul2	yes	DIV.D		M1	Mul1	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Mul1	M2		Val2	Val1	Mul2		

Cycle 16

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6-15	16
SUB.D F8, F6, F2	4	6-7	8
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9-10	11

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	no					
Add2	no					
Add3						
Mul1	no	~~MULT.D~~	~~M2~~	~~Value(F4)~~		
Mul2	yes	DIV.D	Val3	M1	~~Mul1~~	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Val3	M2		Val2	Val1	Mul2		

Cycle 17

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6-15	16
SUB.D F8, F6, F2	4	6-7	8
DIV.D F10, F0, F6	5	17	
ADD.D F6, F8, F2	6	9-10	11

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	no					
Add2	no					
Add3						
Mul1	no					
Mul2	yes	DIV.D	Val3	M1		

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Val3	M2		Val2	Val1	Mul2		

Cycle 56

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6-15	16
SUB.D F8, F6, F2	4	6-7	8
DIV.D F10, F0, F6	5	17-56	
ADD.D F6, F8, F2	6	9-10	11

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	no					
Add2	no					
Add3						
Mul1	no					
Mul2	yes	DIV.D	Val3	M1		

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Val3	M2		Val2	Val1	Mul2		

Cycle 57

指令	指令状态表		
	流出	执行	写结果
L.D F6, 34(R2)	1	2,3	4
L.D F2, 45(R3)	2	3-4	5
MULT.D F0, F2, F4	3	6-15	16
SUB.D F8, F6, F2	4	6-7	8
DIV.D F10, F0, F6	5	17-56	57
ADD.D F6, F8, F2	6	9-10	11

	Load保留站	
	Busy	Address
Load1	no	
Load2	no	
Load3		

部件名称	执行部件保留站					
	Busy	Op	Vj	Vk	Qj	Qk
Add1	no					
Add2	no					
Add3						
Mul1	no					
Mul2	no	~~DIV.D~~	~~Val3~~	~~M1~~		

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	…	F30
部件	Val3	M2		Val2	Val1	Val4		

当采用Tomasulo算法时，在上述给定的时刻，

保留站、load缓冲器以及寄存器状态表中的内容。

指　　令		指令状态表		
		流出	执行	写结果
L.D	F6 , 34(R2)	√	√	√
L.D	F2 , 45(R3)	√	√	
MUL.D	F0 , F2 , F4	√		
SUB.D	F8 , F6 , F2	√		
DIV.D	F10 , F0 , F6	√		
ADD.D	F6 , F8 , F2	√		

名称	保留站						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	yes	LD					45+Regs[R3]
Add1	yes	SUB	Mem[34+Regs[R2]]		Load2		
Add2	yes	ADD			Add1	Load2	
Add3	no						
Mult1	yes	MUL		Reg[F4]	Load2		
Mult2	yes	DIV	Mem[34+Regs[R2]]		Mult1		

	寄存器状态表									
	F0	F2	F4	F6	F8	F10	…	F30		
Qi	Mult1	Load2		Add2	Add1	Mult2	…			

Tomasulo算法具有两个主要的优点：

- 冲突检测逻辑是分布的

  (通过保留站和CDB实现)

  - 如果有多条指令已经获得了一个操作数，并同时在等待同一运算结果，那么这个结果一产生，就可以通过CDB同时播送给所有这些指令，使它们可以同时执行。

- 消除了WAW冲突和WAR冲突导致的停顿

  使用保留站进行寄存器换名，并且在操作数一旦就绪就将之放入保留站。

例2.3 对于例5.2中的代码，假设各种操作的延迟为：

load：1个时钟周期

加法：2个时钟周期

乘法：10个时钟周期

除法：40个时钟周期

给出MUL.D指令准备写结果时各状态表的内容。

解 MUL.D指令准备写结果时各状态表的内容如下图所示。

指　　令		指令状态表		
		流出	执行	写结果
L.D	F6 , 34(R2)	√	√	√
L.D	F2 , 45(R3)	√	√	√
MUL.D	F0 , F2 , F4	√	√	
SUB.D	F8 , F6 , F2	√	√	√
DIV.D	F10, F0, F6	√		
ADD.D	F6 , F8 , F2	√	√	√

与记分牌的对比

指　　令	指令状态表			
	流出	读操作数	执行	写结果
L.D　F6, 34(R2)	√	√	√	√
L.D　F2, 45(R3)	√	√	√	√
MULT.D　F0, F2, F4	√	√	√	
SUB.D　F8, F6, F2	√	√	√	√
DIV.D　F10, F0, F6	√			
ADD.D　F6, F8, F2	√	√	√	

名称	保留站						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	yes	LD					45+Regs[R3]
Add1	yes	SUB	Mem[34+Regs[R2]]		Load2		
Add2	yes	ADD			Add1	Load2	
Add3	no						
Mult1	yes	MUL		Reg[F4]	Load2		
Mult2	yes	DIV	Mem[34+Regs[R2]]		Mult1		

	寄存器状态表								
	F0	F2	F4	F6	F8	F10	…	F30			
Qi	Mult1	Load2		Add2	Add1	Mult2	…				

MUL.D指令准备写结果时各状态表的内容

名称	保留站						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	no						
Add1	no						
Add2	no						
Add3	no						
Mult1	yes	Mul	Mem[45+Regs[R3]]	Reg[F4]			
Mult2	yes	DIV		Mem[34+Regs[R2]]	Mult1		

	寄存器状态表								
	F0	F2	F4	F6	F8	F10	...	F30	
Qi	Mult1			Mult2	...				

2.3.3.3 具体算法

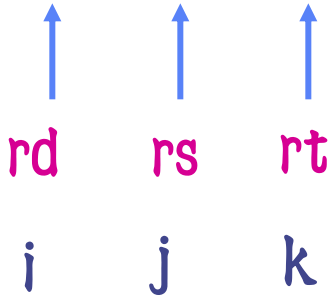
各符号的意义

- r：分配给当前指令的保留站或者缓冲器单元编号；
- rd：目标寄存器编号；
- rs、rt：操作数寄存器编号；
- imm：符号扩展后的立即数；
- RS：保留站；
- result：浮点部件或load缓冲器返回的结果；
- Qi：寄存器状态表；
- Regs[ ]：寄存器组；

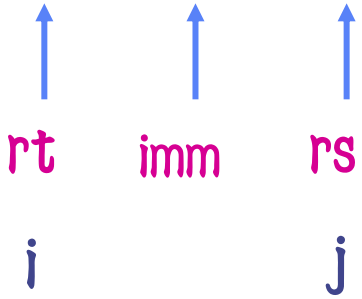
- 与rs对应的保留站字段：Vj，Qj
- 与rt对应的保留站字段：Vk，Qk
- Qi、Qj、Qk的内容或者为0，或者是一个大于0的整数。
  - Qi为0表示相应寄存器中的数据就绪。
  - Qj、Qk为0表示保留站或缓冲器单元中的Vj或Vk字段中的数据就绪。
  - 当它们为正整数时，表示相应的寄存器、保留站或缓冲器单元正在等待结果。

符号说明：　(举例)

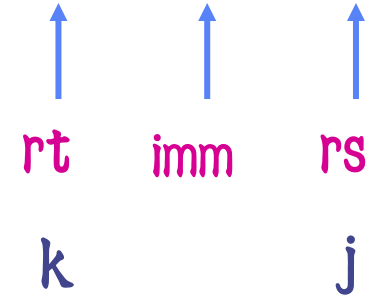
MUL.D　F4, F0, F2



L.D F2, 45 (R3)

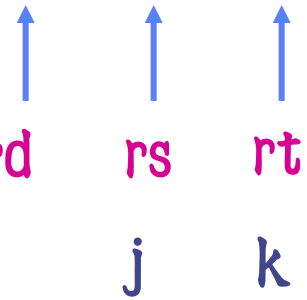


S.D F3, 40 (R4)



1. 指令流出

MUL.D F4, F0, F2



- 浮点运算指令

进入条件：有空闲保留站（设为r）

操作和状态表内容修改：

if (Qi[rs] ≠ 0) //检测第一操作数是否就绪

{ RS[r].Qj ← Qi[rs] }; //第一操作数没有就绪，进行寄存器换名，即把将产生该操作数的保留站的编号放入当前保留站的Qj。该编号是一个大于0的整数。

else { RS[r].Vj ← Regs[rs] ; //第一操作数就绪。把寄存器rs //中的操作数取到当前保留站的Vj。

RS[r].Qj ← 0 } //置Qj为0，表示当前保留站的Vj中的操作数就绪。

if (Qi[rt] ≠ 0) // 检测第二操作数是否就绪

{ RS[r].Qk ← Qi[rt] ; //第二操作数没有就绪，进行寄存器换名，即把将产生该操作数的保留站的编号放入当前保留站的Qk。该编号是一个大于0的整数。

else { RS[r].Vk ← Regs[rt] ; //第二操作数就绪。把寄存器rt中的操作数取到当前保留站的Vk。

RS[r].Qk ← 0 } // 置Qk为0，表示当前保留站的Vk中的操作数就绪。

RS[r].Busy ← yes; //置当前保留站为“忙”

RS[r].Op ← Op; //设置操作码

Qi[rd] ← r; // 把当前保留站的编号r放入rd所对应的寄存器状态表项，以便rd将来接收结果。

➢ load和store指令

L.D F2, 45 (R3)

rt imm rs

i j

进入条件：缓冲器有空闲单元（设为r）

操作和状态表内容修改：

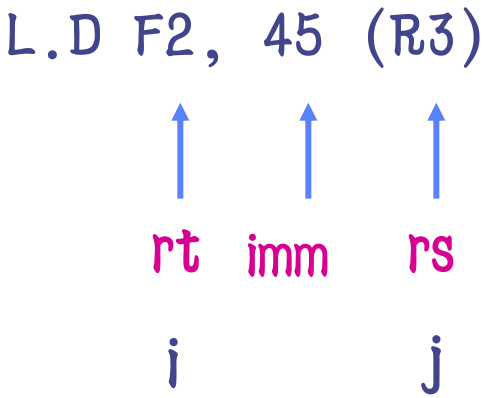
if (Qi[rs] ≠ 0) // 检测第一操作数是否就绪

{RS[r].Qj ← Qi[rs] } //第一操作数没有就绪，进行寄存器换名，即把将产生该操作数的保留站的编号存入当前缓冲器单元的Qj。

else

{RS[r].Vj ← Regs[rs]; // 第一操作数就绪，把寄存器rs中的
// 操作数取到当前缓冲器单元的Vj

RS[r].Qj ← 0 }; // 置Qj为0，表示当前缓冲器单元的Vj
// 中的操作数就绪。



RS[r].Busy ← yes; // 置当前缓冲器单元为“忙”

RS[r].A ← Imm; // 把符号位扩展后的偏移量放入
// 当前缓冲器单元的A

对于load指令:

Qi[rt] ← r; // 把当前缓冲器单元的编号r放入
// load指令的目标寄存器rt所对应的寄存器
// 状态表项，以便rt将来接收所取的数据。

S.D F3, 40 (R4)

rt imm rs

k j

对于store指令：

if (Qi[rt] ≠ 0) // 检测要存储的数据是否就绪

{RS[r].Qk ← Qi[rt] } //该数据尚未就绪，进行寄存器换名，

即把将产生该数据的保留站的编号放入当前缓冲器单元的Qk。

else

{RS[r].Vk ← Regs[rt]; // 该数据就绪，把它从寄存器rt取到

// store缓冲器单元的Vk

RS[r].Qk ← 0 }; // 置Qk为0，表示当前缓冲器单元的Vk

// 中的数据就绪。

1. 执行

- 浮点操作指令
  - 进入条件：（RS[r].Qj = 0）且（RS[r].Qk= 0）； // 两个源操作数就绪
  - 操作和状态表内容修改：进行计算，产生结果 。
- load/store指令
  - 进入条件：（RS[r].Qj = 0）且r成为load/store缓冲队列的头部
  - 操作和状态表内容修改：

RS[r].A ← RS[r].Vj + RS[r].A; //计算有效地址

对于load指令，在完成有效地址计算后，还要进行：

从Mem[RS[r].A]读取数据; //从存储器中读取数据

1. 写结果

- **浮点运算指令和load指令**

进入条件：保留站r执行结束，且CDB就绪。

操作和状态表内容修改：

```
∀x  (if  (Qi[x] = r)                  // 对于任何一个正在等该结果的浮点寄存器x
    { Regs[x] ← result;               // 向该寄存器写入结果
      Qi[x] ← 0 };                    // 把该寄存器的状态置为数据就绪
∀x  (if  (RS[x].Qj = r)               // 对于任何一个正在等该结果作为第一操作数的保留站x
   {RS[x].Vj ← result;                // 向该保留站的Vj写入结果
    RS[x].Qj ← 0 };                   // 置Qj为0，表示该保留站的Vj中的操作数就绪
```

∀x （if （RS[x].Qk = r）　　// 对于任何一个正在等该结果作为第二操作数的保留站x

{RS[x].Vk ← result;　　// 向该保留站的Vk写入结果

RS[x].Qk ← 0 }；　　// 置Qk为0，表示该保留站的Vk中的操作数就绪。

RS[r].Busy ← no;　　// 释放当前保留站，将之置为空闲状态。

- store指令

进入条件：保留站r执行结束，且RS[r].Qk = 0　　// 要存储的数据已经就绪

操作和状态表内容修改：

Mem[RS[r].A] ← RS[r].Vk　　// 数据写入存储器，地址由store

// 缓冲器单元的A字段给出。

RS[r].Busy ← no;　　//释放当前缓冲器单元，将之置为空闲状态。

1. Tomasulo算法的缺陷是需要大量的硬件
   - 需要大量高速运转的存储器件及复杂的控制逻辑，特别是每个保留站都要有一个相关联的逻辑复杂的高速缓存
   - 使用单条完整的总线（CDB），所以反过来会限制性能的提高。即使可以增加CDB，但是增加CDB即意味着需要增加与流水线中所有硬件的连接，包括保留站
2. 对于单发射流水线，Tomasulo算法比起编译器调度所带来的有利之处，则会少于实现起来所需要的成本的增加
   - 但是随着处理机在发射能力方面的提高及代码复杂度的增加，将使寄存器换名、动态调度、推测执行技术变得越来越重要
3. 对指令执行存在一个限制，如果流水中存在分支指令未完成，当前指令不能进入“执行”



授课教师：王强
邮箱：qiang.wang@hit.edu.cn

谢谢

Thank You